



INSTITUTO POLITÉCNICO
NACIONAL



UNIDAD PROFESIONAL
INTERDISCIPLINARIA EN INGENIERÍA
Y TECNOLOGÍAS AVANZADAS

MULTIMEDIA

PRÁCTICA 1: ESTEGANOGRAFÍA LSB EN IMÁGENES BMP

ALUMNO: HERNÁNDEZ AVILÉS LUIS ALBERTO

BOLETA: 2023640387

GRUPO: 3TM2

PROFESOR: SIERRA ROMERO NOE

Objetivo general: Implementar desde cero la técnica de ocultamiento de información mediante sustitución del bit menos significativo (Least Significant Bit – LSB) en imágenes BMP de 24 bits por píxel.

Competencias por desarrollar: Manipulación de bits, diseño de algoritmos de codificación/decodificación, análisis de capacidad de almacenamiento y evaluación de calidad de imagen (PSNR).

Herramientas sugeridas: Python 3, imagen BMP de 24 bpp de al menos 200x200 px.

Entregables:

Tabla comparativa de resultados:

Imagen (px)	Tamaño del mensaje	PSNR obtenido	¿Imperceptible?
200 x 200	50 bytes	≈ 80.20 dB	Sí
200 x 200	500 bytes	≈ 83.74 dB	Sí
512 x 512	5,000 bytes	≈ 88.20 dB	Sí
512 x 512	Capacidad máx.	≈ 51.14 dB	Sí

Código:

```
import struct
import math
import os

def leer_bmp(filepath):
    """Retorna (header_bytes, pixels, width, height, row_size)"""
    with open(filepath, 'rb') as f:
        data = f.read()

    offset = struct.unpack_from('<I', data, 10)[0]
    width = struct.unpack_from('<i', data, 18)[0]
    height = struct.unpack_from('<i', data, 22)[0]

    row_size = (width * 3 + 3) & ~3
    header = bytearray(data[:offset])
    pixels = bytearray(data[offset:])
    return header, pixels, width, height, row_size

def guardar_bmp(filepath, header, pixels):
    """Guarda los bytes en un nuevo archivo BMP"""
    with open(filepath, 'wb') as f:
        f.write(header)
        f.write(pixels)

def embed_lsb(src_path, dst_path, mensaje):
    """Oculta un mensaje usando la técnica LSB"""
```

```

header, pixels, width, height, row_size = leer_bmp(src_path)
msg_bytes = mensaje.encode('utf-8')
msg_len = len(msg_bytes)

# Encabezado de 32 bits con la longitud + mensaje
datos = struct.pack('<I', msg_len) + msg_bytes

bits = []
for byte in datos:
    for i in range(7, -1, -1):
        bits.append((byte >> i) & 1)

if len(bits) > len(pixels):
    raise ValueError('Mensaje demasiado largo para esta imagen') [cite: 78, 127]

pixels_mod = bytearray(pixels)
for idx, bit in enumerate(bits):
    pixels_mod[idx] = (pixels_mod[idx] & 0xFE) | bit

guardar_bmp(dst_path, header, pixels_mod)
print(f"[OK] Mensaje de {msg_len} bytes incrustado en {dst_path}")

def extract_lsb(stego_path):
    """Recupera el mensaje oculto en la imagen"""
    header, pixels, width, height, row_size = leer_bmp(stego_path)

    # 1. Leer los primeros 32 bits (longitud)
    len_bits = [pixels[i] & 1 for i in range(32)]

    # 2. Reconstruir esos bits en 4 bytes
    len_bytes = bytearray()
    for i in range(0, 32, 8):
        byte = 0
        for bit in len_bits[i:i+8]:
            byte = (byte << 1) | bit
        len_bytes.append(byte)

    # 3. Desempaquetar respetando el formato Little Endian ('<I')
    msg_len = struct.unpack('<I', len_bytes)[0]

    # 4. Calcular los bits totales a leer
    total_bits = 32 + (msg_len * 8)

    # Control de seguridad extra para evitar el IndexError
    if total_bits > len(pixels):
        return f"Error: La longitud extraída ({msg_len} bytes) excede la imagen."

    # 5. Extraer los bits del mensaje y reconstruirlos
    msg_bits = [pixels[i] & 1 for i in range(32, total_bits)]

    msg_bytes = bytearray()
    for i in range(0, len(msg_bits), 8):
        byte = 0
        for bit in msg_bits[i:i+8]:
            byte = (byte << 1) | bit
        msg_bytes.append(byte)

    return msg_bytes.decode('utf-8')

```

```

def calcular_psnr(original_path, stego_path):
    """Calcula MSE y PSNR para evaluar la degradación visual"""
    header_o, pix_orig, w, h, rs = leer_bmp(original_path)
    header_s, pix_steg, _, _, _ = leer_bmp(stego_path)

    # Error Cuadrático Medio (MSE)
    mse = sum((a - b)**2 for a, b in zip(pix_orig, pix_steg)) / (w * h * 3)

    if mse == 0:
        return 0.0, float('inf')

    # Relación señal-ruido de pico (PSNR)
    psnr = 10 * math.log10(255**2 / mse)
    return mse, psnr # Retorna ambos explícitamente

# --- PRUEBAS DE LABORATORIO ---
if __name__ == "__main__":
    IMG_200 = 'Img200.bmp'
    IMG_512 = 'Img512.bmp'

    # --- PRUEBA 1: 200x200 ---
    if os.path.exists(IMG_200):
        print("\n--- Ejecutando Prueba 1 (200x200) ---")
        STEGO_200 = 'stego_200.bmp'
        embed_lsb(IMG_200, STEGO_200, "PRUEBA 200 PX")
        msg_1 = extract_lsb(STEGO_200)
        mse_1, psnr_1 = calcular_psnr(IMG_200, STEGO_200)
        # Código que ayuda en eliminar el texto que causaba el NameError
        print(f"Mensaje: {msg_1}")
        print(f"MSE: {mse_1:.6f} | PSNR: {psnr_1:.2f} dB")

    # --- PRUEBA 2: 512x512 ---
    if os.path.exists(IMG_512):
        print("\n--- Ejecutando Prueba 2 (512x512) ---")
        STEGO_512 = 'stego_512.bmp'
        embed_lsb(IMG_512, STEGO_512, "MENSAJE PARA LA IMAGEN GRANDE UPIITA")
        msg_2 = extract_lsb(STEGO_512)
        mse_2, psnr_2 = calcular_psnr(IMG_512, STEGO_512)
        print(f"Mensaje: {msg_2}")
        print(f"MSE: {mse_2:.6f} | PSNR: {psnr_2:.2f} dB")
    else:
        print(f"\nNo se encontró {IMG_512}")

    # --- PRUEBA 3: Capacidad Máxima (512x512) ---
    if os.path.exists(IMG_512):
        print("\n--- Ejecutando Prueba 3 (Capacidad Máxima 512x512) ---")
        STEGO_MAX = 'stego_max.bmp'

        # Creamos un mensaje de 98,300 caracteres
        mensaje_max = "A" * 98300

        try:
            embed_lsb(IMG_512, STEGO_MAX, mensaje_max)
            msg_rec = extract_lsb(STEGO_MAX)
            mse_max, psnr_max = calcular_psnr(IMG_512, STEGO_MAX)

            print(f"Mensaje recuperado con éxito (Longitud: {len(msg_rec)})")
            print(f"MSE: {mse_max:.6f} | PSNR: {psnr_max:.2f} dB")
            print("--- PRUEBAS FINALIZADAS ---")
        except:
            pass

```

```
except ValueError as e:  
    print(f"Error esperado: {e}")
```

Preguntas de análisis:

1. ¿Qué sucede si se intenta ocultar un mensaje más largo que la capacidad de la imagen? Implemente un control de error apropiado.

Si se llegará a intentar ocultar más bits de los que actualmente la imagen o las imágenes de prueba tienen, simplemente el programa intentaría escribir en direcciones de memoria que no existen o sobrescribiría datos críticos de la imagen, corrompiéndola y comprometiendo a este significativamente.

Un control de error apropiado ante esta práctica sería una validación que muestre que si la longitud de bits es menor que la longitud de píxeles muestre un "raise ValueError()", deteniendo el programa de una manera confiable e informa al usuario que la capacidad ha sido excedida antes de intentar modificar el bytearray.

2. Compare visualmente (o mediante histograma) la imagen original y la esteganografiada. ¿Hay diferencias observables en el histograma de intensidades?

A simple vista la diferencia es bastante nula, de igual manera si a una persona con nulo conocimiento sobre la técnica esteganográfica que se llevo a cabo se le muestra ambas imágenes. Probablemente la respuesta sería que ambas imágenes son completamente iguales.

3. Proponga una estrategia para aumentar la capacidad de ocultamiento a 2 LSBs por canal. ¿Cómo afecta esto al PSNR?

Podría ser que en lugar de usar una máscara 0xFE para limpiar un bit, usaríamos 0xFC (que en binario es 11111100) para limpiar los dos bits menos significativos. Luego, tomaríamos el mensaje y lo dividiríamos en grupos de 2 bits para insertarlos en cada byte de color. Afectando al PSNR, este disminuiría considerablemente.

4. ¿Por qué el formato BMP es preferible al JPEG para esteganografía LSB? ¿Qué ocurriría si se guarda la imagen stego como JPEG?

Se prefiere el formato BMP por ser un formato de mapa de bits sin pérdida alguna, garantizando que cada bit modificado se mantendrá de la misma forma al momento de guardar el archivo.

Si la imagen se llegará a guardar como JPEG, esta redondearía y descartaría variaciones pequeñas de color (precisamente los LSB) para ahorrar espacio ya que usa una compresión con pérdida. Resultando en una destrucción de los bits del mensaje oculto.

Conclusiones:

En general puedo añadir que luego de la realización de esta práctica en donde se implementaron conocimientos en esteganografía fueron totalmente reveladores, ya que si bien, anteriormente ya había escuchado informalmente acerca de esta técnica, no sabía exactamente como implementarla y llevarla a cabo. Ahora conozco y soy capaz de aplicar una técnica esteganográfica en una imagen con el formato BMP, auxiliándome de la técnica del bit menos significativo para poder almacenar mensajes ocultos dentro de imágenes, integrándolo en un script de lenguaje python.

Imágenes:

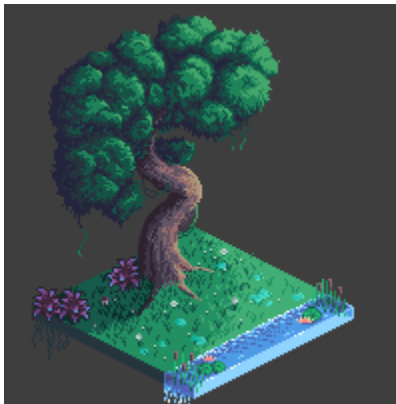


Ilustración 1. Imagen original 200x200

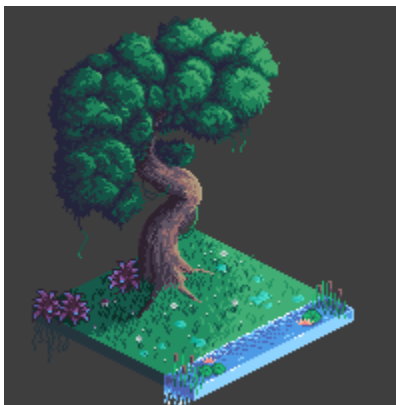


Ilustración 2. Imagen esteganografiada 200x200

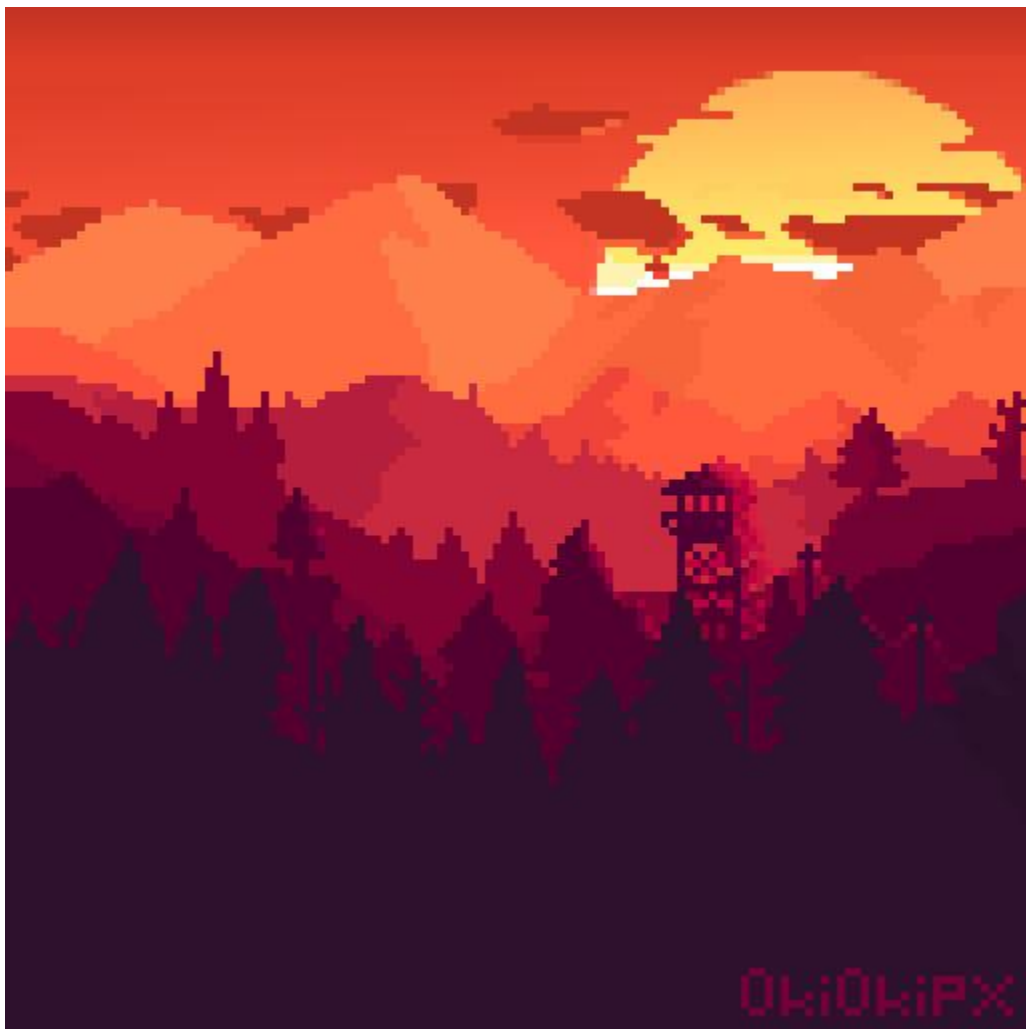


Ilustración 3. Imagen original 512x512

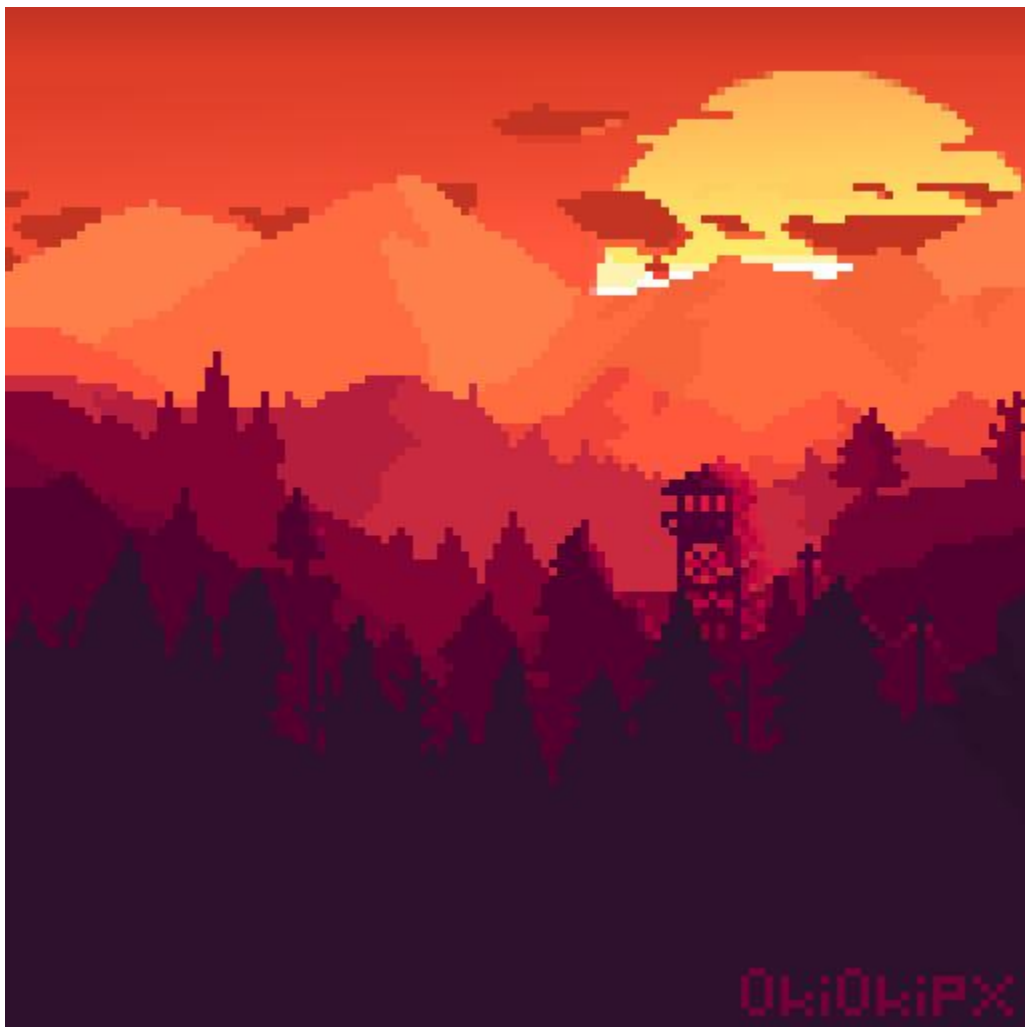


Ilustración 4. Imagen esteganografiada 512x512

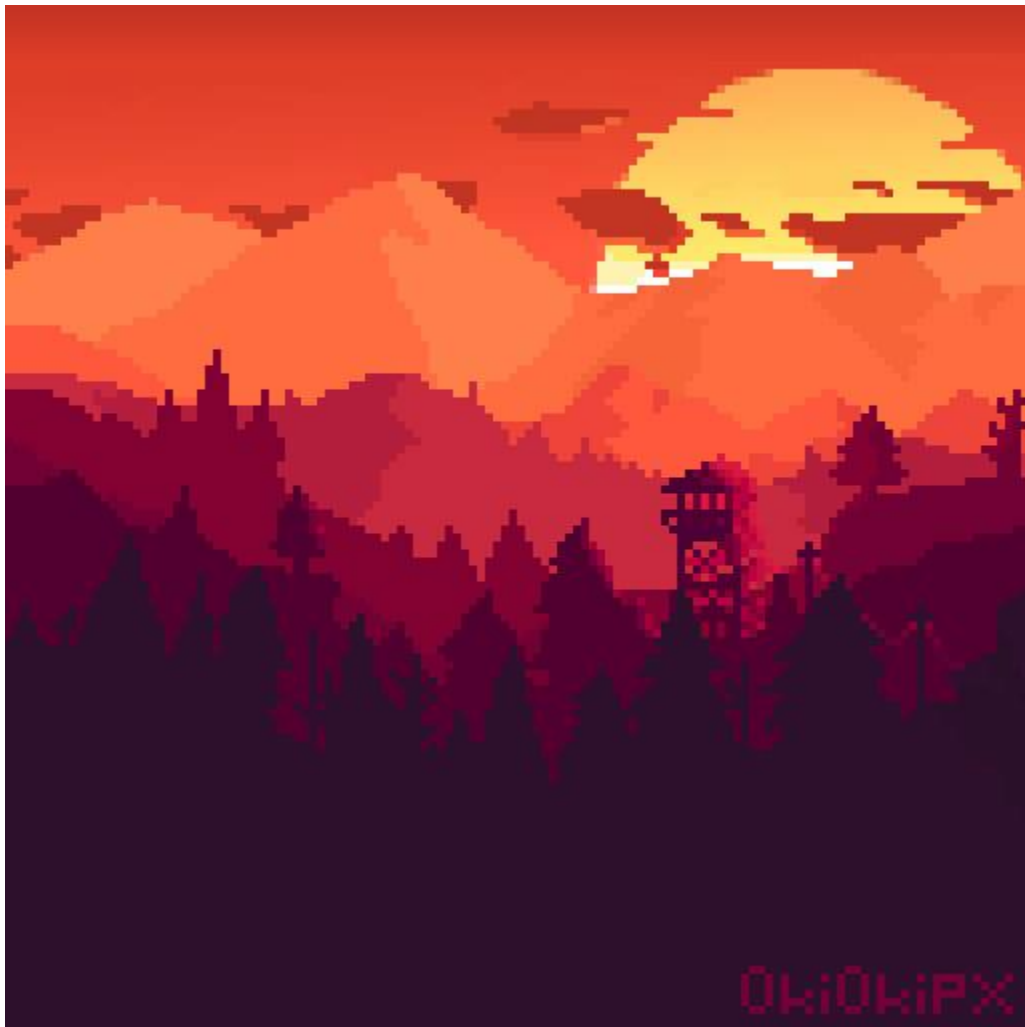


Ilustración 5. Imagen esteganografiada máxima